TEXAS A&M UNIVERSITY

Predicate logic

CSCE 222: Discrete Structures for Computing

Predicates and quantifiers

Predicate logic introduces two concepts:

Predicates

Quantifiers

Statements have two parts:

1. a **variable**
2. a **predicate:** a property that the variable might or might not satisfy

Predicates

Example: “x is greater than 3”

P(x) is the statement “x is greater than 3”
P is the propositional function
x is the variable
“greater than 3” is the predicate.

Predicates

Suppose $P(x)$ is "$x > 3$"

$P(4)$ is true, since $4 > 3$

$P(2)$ is false, since $2 \leq 3$

Suppose $Q(x, y)$ is "$x = y + 3$"

$Q(1,2)$ is false, since $1 \neq 2 + 3$

$Q(3,0)$ is true, since $3 = 0 + 3$

Predicates in programming

Swap variables x and y.

Suppose x = a and y = b. We want to show that after the code runs, x = b and y = a.

```
temp ← x
x ← y
y ← temp
```

Predicates in programming

Precondition: predicate states what is true before the code runs. P(x, y) = "x = a and y = b"

Post condition: predicate stating what is true after the code runs. Q(x, y) = "x = b and y = a"

x = a, y = b (i.e., P(x, y) is true)

1. temp ← x // x= a, temp= a, y= b

2. x ← y // x= b, temp= a, y= b

3. y ← temp // x= b, y= a (i.e., Q(x,y) is true)

Quantifiers

Sometimes we consider the value of a predicate for multiple values of the variable(s). To specify a range of values for the variable(s), we use quantifiers.

Universal quantifier: ($\forall$)

Indicates the predicate should hold for all elements of the domain.

Existential quantifier: ($\exists$)

Indicates the predicate should hold for at least one element of the domain.

Universal quantifier (∀)

Let **P(x)** be a predicate

Universal quantifier: ∀x P(x)

∀x P(x) is **true** if P(x) is true for **all x** in the domain

∀x P(x) is **false** if P(x) is false for **at least one** x in the domain

Universal quantifier (∀)

Example 1

P(x) is "x + 1 > x" (Domain: real numbers)

∀x P(x) is true. since x + 1 > x **for every** real number x

Example 2

Q(x) is "x < 2" (Domain: real numbers)

∀x Q(x) is false. since **there exists** a real number that is ≥ 2

Existential quantifier (∃)

Let P (x) be a predicate

Existential quantifier: ∃ x P(x)

- **∃ x P(x)** is **true** if P(x) is true for **at least one** x in the domain.
- **∃ x P(x)** is **false** if P(x) is false **for every x** in the domain.

Existential quantifier (∃)

Example 1

Q(x) is "x < 2" (Domain: real numbers)

∃ x Q(x) is true since **there exists** a real number that is < 2

Example 2

R(x) is "x > x + 1" (Domain: real numbers)

∃ x R(x) is false since there exists no real number that is larger than 1 plus itself

Logical Equivalences and Quantifiers

Statements involving predicates and quantifiers are logically equivalent iff they have the same truth value.

The universal quantifier distributes over a conjunction

$$\forall x\,(P(x) \land Q(x)) \equiv \forall x\, P(x) \land \forall x\, Q(x)$$

The existential quantifier distributes over a disjunction

$$\exists x\,(P(x) \lor Q(x)) \equiv \exists x\, P(x) \lor \exists x\, Q(x)$$

The universal quantifier does not distribute over a disjunction!

The existential quantifier does not distribute over a conjunction!

Negating Quantified Expressions

What is the negation of "Every student in the room has taken Calculus 1"?
$\neg \forall x\, P(x)$ is logically equivalent to $\exists x\, \neg P(x)$

What is the negation of "At least one student in the room has taken Calculus 1"?
$\neg \exists x\, P(x)$ is logically equivalent to $\forall x\, \neg P(x)$

Applications of Quantified Expressions

Formalizing system specifications:

"Every email message larger than one megabyte will be compressed."

"If a user is active, at least one network link will be available."

Nested Quantifiers

We can build complex logical statements by nesting quantifiers.

$\forall x\ \exists y\ (x + y = 0)$. For every x, there exists y such that $x + y = 0$. ($y = -x$)

$\forall x\ \forall y\ (x + y = y + x)$. The commutative law of addition.

$\exists x\ \forall y\ (x \cdot y = 0)$. There exists x such that for every y, $x \cdot y = 0$. ($x = 0$)

Order of Nested Quantifiers

The order of two universal quantifiers does not matter:

$\forall x \forall y\, P(x, y) \equiv \forall y \forall x\, P(x, y)$

The order of two existential quantifiers does not matter:

$\exists x \exists y\, P(x, y) \equiv \exists y \exists x\, P(x, y)$

Order of Nested Quantifiers

The order of a universal quantifier and an existential quantifier does matter:

$\forall x\ \exists y\ P(x, y) \not\equiv \exists x\ \forall y\ P(x, y)$

$\forall x\ \exists y\ (x + y = 0)$ is true

$\exists x\ \forall y\ (x + y = 0)$ is false!

Negating Nested Quantifiers

It is convenient to rewrite the expression so that negations only occur immediately before predicates, and not before quantifiers.

Proceed step-by-step!

$$
\begin{aligned}
\neg(\forall x\ \exists y\ (x \cdot y = 1)) &\equiv \exists x\ \neg(\exists y\ (x \cdot y = 1)) \\
&\equiv \exists x\ \forall y\ \neg(x \cdot y = 1)) \\
&\equiv \exists x\ \forall y\ (x \cdot y \neq 1)
\end{aligned}
$$

Precedence of Quantifiers

The quantifiers $\forall$ and $\exists$ have higher precedence than all the logical operators.

e.g. $\forall x\, P(x) \vee Q(x)$ means $\mathbf{(\forall x\, P(x))} \vee Q(x)$

$\forall x\, (P(x) \vee Q(x))$ means something different!

Predicates in Python

Concept	Notation	Python	Meaning
Existential Quantifier	$\exists x\, P(x)$	`any(P(x) for x in users)`	At least one is True
Universal Quantifier	$\forall x\, P(x)$	`all(P(x) for x in users)`	All are True
Negation of $\exists$	$\neg\exists x\, P(x)$	`not any(P(x) for x in users)`	None are True
Equivalent	$\forall x\, \neg P(x)$	`all(not P(x) for x in users)`	All are False

Example

Negate:

$\forall x\, Q(x) \wedge \exists y\, P(y)$

Binding of Variables

When a quantifier is used on a variable, it is bound.

If a variable is not bound or set to a particular value, it is free.

e.g. In $\exists x\,(x + y = 1)$, x is bound, y is free